TypeScript Without Surprises: Smarter Error Handling with Effect

12/9/2025, 9:16:21 PM

1/41 TypeScript Without Surprises: Smarter Error Handling with Effect

TypeScript Without Surprises: Smarter Error Handling with Effect

---

2/41 TypeScript Without Surprises: Smarter Error Handling with Effect

This talk will hopefully change the way you think about handling errors in TypeScript. We'll explore the Effect library - and see how it solves problems you might not even realize you have.

3/41 About me

My name is Nir. Tamir. I've been doing frontend for over a decade. You can find more about me at nirtamir.com.

4/41 TypeScript is great

TypeScript is great - It helps us catch errors before they happen.

5/41 TypeScript is great

If a function expects a number and we pass a string, we get compile time error.

6/41 TypeScript is great

And when we fix it, TypeScript infers the return type automatically. This makes composition easy and refactoring safe.

7/41 When the type system can't help

In real apps, we hit edge cases the type system can't catch. Dividing by zero, for example, returns Infinity. Its a valid number - so TypeScript accepts it - but it's probably not what we want.

8/41 Errors are typed as 'unknown'

We can throw an error and catch it. But notice - the error is typed as unknown. TypeScript has no idea what this error looks like. We have to manually check its shape.

9/41 The invisible throw problem

Here's an even bigger problem. Look at this function. 1 TypeScript tells us what parameters this function needs and how to call it. 2 But this function throws an error - and TypeScript doesn't tell us

There's no "throws" annotation in TypeScript. We have to read the implementation or just find out at runtime. In production.

10/41 How do you handle an error you can't see?

Think about it for a moment.

If a function doesn't tell you what can go wrong, how do you write proper error handling?

Do you wrap everything in try-catch and return a generic error?

This reminds me of the days of JavaScript before TypeScript. Remember that world?

You couldn’t trust function parameters or return values. You had to dig into the code just to see what it returned. And always watch out for undefined and null.

We had so many bugs – just because we didn’t know what could be null.

TypeScript solved that problem beautifully. Now the compiler tells us exactly what a function expects – and what it returns.

But… TypeScript doesn’t tell us what errors a function can throw.

Even when we know something might fail, the catch block gives us only unknown.

So we’re back to the old JavaScript problems – but this time, with errors.

No type safety. No compiler help. Just runtime surprises.

We can’t handle errors well if we don’t know they exist – or what they are.

11/41 How we handle errors today

So how we handle errors in TypeScript?

12/41 Custom error classes

One option is to define a custom error that extends error, with a custom tag. 1 2 This lets us throw a specific type of error instead of a generic one. 3 That way, we can check the type inside the catch block and handle it accordingly. 4 it’s still typed as unknown, so we need a type guard like instanceof. This works, but it’s manual and easy to forget or get wrong.

13/41 Return errors as Values

In languages like Go, there's no concept of throwing errors – you return them instead. We can do something similar in TypeScript. 1 Here, the Result type makes sure we either get data or error, but not both.

2 TypeScript even infers the structure for us, so we can easily pattern match or check which case we're in. And the nice part is, it makes the function's possible errors explicit in the type system. But it has a lot of repetitive code.

14/41 Wrapping and Unwrapping Gets Messy

1 Composing multiple such functions gets messy, since we now have to wrap and unwrap manually.

15/41

You can ask any Go developers if this looks familiar…

16/41 TypeScript is great

So TypeScript is great - for the happy path. When everything goes right, the types guide us perfectly. But when things go wrong? We're alone.

[PAUSE]

17/41 What we're missing

What we really need is a way to make errors part of the type system. 1 So the compiler can help us handle them, just like it helps us with success values. That's where Effect comes in.

18/41 [Effect](https://effect.website/docs/getting-started/introduction/)

Effect is a powerful TypeScript library that brings errors into the type system. It makes every possible failure explicit and gives you tools to handle them elegantly.

19/41 The Effect type

The Effect type has three parts: Success - what value we get when things work Error - what can go wrong Requirements - what dependencies we need

---

20/41 Effect Values

We can create effects using Effect.succeed for successful values, or Effect.fail for errors. This avoids throwing exceptions and keeps errors explicit and typed. It looks very similar to Promise.resolve & Promise.reject.

Effect.succeed creates an effect that resolves with the number 42. So its type is: Effect<number, never, never> — meaning it produces a number, and it never fails or depends on anything.

Effect.fail creates an effect that fails with the string "oops". So its type is: Effect<never, string, never>.

21/41 Effects are blueprints

An Effect is like a blueprint - it describes what to do, but doesn't do it. We have to call `runSync` or `runPromise` to run the effect.

This separation lets us compose, transform, and handle errors before anything actually runs.

[PAUSE]

22/41 A Real Example

Let's build something real with what we learned.

1 First, we define a custom error type. This is a tagged error that Effect can recognize.

2 Now, we define a program using `Effect.gen()`.
It accepts a generator function, so we write `function*`.

3 If isCreditExceeded, we yield* Effect.fail with our typed error.

The yield* works just like await. It unwraps the Effect and gives you the success value — so you can use it like a normal variable.

But here's the magic: if any Effect fails, the error bubbles up automatically. The errors just accumulate in the type signature.

yield* here is what propagates the error up.

Otherwise, we just return the result.

4 Now the function signature is honest. It tells us exactly what can happen. No surprises.

23/41 Error in the type

If we run this Effect directly, it can throw. We're back to the same problem - an unhandled error at runtime. But now we KNOW it's there because the type told us.

24/41 Error handled in the type

Here's where Effect shines.

1 We use catchTag to handle specific errors by their tag. TypeScript autocompletes "PaymentFailed" because it's in the type. When it happens, we recover by returning a different string value. But now it's explicit and intentional.

2 Look at the new type: Effect<string, never, never> The error is GONE.

3 This is the key insight: we decide WHEN and WHERE to handle errors. And the type system keeps perfect track of what's handled and what's not.

[PAUSE - let this land]

25/41 The type guides us

Before we handle the error, the type shows it's there. 1 After we handle it, the error becomes 'never'— meaning zero errors remain.

26/41 Real programs have many failures

Real programs have multiple failure modes.

1 We might have network errors, validation errors, database errors, and so on.

2 When we compose Effects, the errors accumulate in the type. NetworkError OR ValidationError - both are tracked.

3 validateUser is a function my teammate wrote - I don't need to took at the implementation to see what it throws.
Each yield may introduce a new type of error, which the compiler then collects.

4 Notice that the code focuses on the happy path, but every possible failure is visible.

27/41 Handling Multiple Errors

We can handle multiple errors at once with catchTags.

1

2 For NetworkError, we fall back to a cached user. 3 For ValidationError, we transform it into a different error type.

4

This is powerful - we can recover from some errors and transform others. The type system tracks everything.

5 After handling, NetworkError and ValidationError are gone from the type. We're left with only BadRequestError.

6 We can't forget or ignore errors - they're right there in the signature.

28/41 Before Effect

Here's typical TypeScript. Three operations, each might fail, but the function signature doesn't tell us anything. We have to read the implementation, hope for documentation, or just cross our fingers.

29/41 After Effect

With Effect, the function signature tells the complete story. 1 Every possible error is tracked in the type: FetchError, ParseError, SaveError.

2 Inside Effect.gen, we write almost like normal code. yield* unwraps the Effect and gives us the value If any step fails, the error propagates automatically The compiler knows about all of them

3 Happy path code - with No surprises and no hidden failures. This is the key difference.

[PAUSE - this is important]

30/41 The difference

Look at the difference 1 It's almost identical

31/41 We can use the type system to track **errors**, not only **success** values

This is the core insight.

TypeScript already tracks success values beautifully. Effect extends that same power to errors.

[PAUSE - this is the thesis of the talk]

32/41

This is what we're trying to avoid. Generic errors that tell us nothing. They hide the real problem and make debugging impossible.

[PAUSE]

Effect makes every error specific and actionable.

33/41 From error tracking to recovery

Up until now, we've been tracking errors.

We saw how the type system keeps a full record of all possible failures — which ones happen, which ones remain after we recover.

But tracking is just the beginning.

1

Once we know what can fail, we can start composing workflows in all sorts of ways: retrying operations, repeating effects, scheduling tasks, and more.

Let's take a look at that next with few examples.

34/41 [Beyond Error Handling:](https://effect.kitlangton.com/) Timeouts

You can add timeouts to any Effect. If it takes too long, it fails with a TimeoutError.

We have orderDelivery() and we don't want to wait forever. Using Effect.timeout(pizza, "1 second"), the effect fails if it takes too long — the pizza gets cold.

1 After one second, it fails automatically with a timeout.

35/41 [Beyond Error Handling:](https://effect.kitlangton.com/) Retries

swipeCard() might fail temporarily. With Effect.eventually(swipeCard), it keeps retrying until it succeeds, ignoring errors.

1 Each attempt runs automatically until the card is accepted.

36/41 [Beyond Error Handling:](https://effect.kitlangton.com/) Retry with schedules

We try to attemptToWakeUp() on a schedule: every 2 seconds, up to 4 times. Using Effect.retry(wakeUp, snoozeSchedule), the effect retries according to the schedule.

1 It stops once it succeeds or reaches the limit.

37/41 [Beyond Error Handling:](https://effect.kitlangton.com/) Exponential backoff

For attemptParallelPark(), we retry with exponential backoff: Effect.retry(park, Schedule.exponential("700 millis")).

1 Each retry waits longer than the last, reducing pressure and avoiding too-frequent attempts.

38/41 More errors - better visibility

At Vercel, they had a feature for auto-renewing domains — and lots of mysterious issues. After switching to Effect (and neverthrow) and similar concepts, suddenly the error count jumped — from 3 1 to 17. But that was actually good news — it meant they weren't hiding problems anymore. They could finally see what was really happening. And when you know your errors you can decide how to handle them - by recover from them with retries mechanism, transforming them, or giving the user more context about what fails.

It becomes much easier to find the root cause of an error instead of just seeing a generic one — and since it's still TypeScript, it doesn't force us into a new ecosystem.

Effect might look different at first, with its generators and yield, but it fits naturally once you get used to it. The key idea is: you don't have to handle errors immediately — just make sure you're aware of them and don't ignore them.

39/41 Key Takeaways

Let's recap what we've learned.

1 TypeScript gives us amazing safety for successful operations.

2 But when things fail, we lose that safety. Errors are invisible and untyped.

3 Effect brings errors into the type system, making them visible and trackable. You can focus on the happy path and the compiler won't let you ignore errors. You decide when and how to handle them, but you can't forget them.

4 And paradoxically, having more specific errors makes your code more reliable. Because you can see and handle each failure mode appropriately.

[PAUSE - let them absorb this]

40/41

Thank you for your time! The slides are available at nirtamir.com And I highly recommend checking out effect.website - the docs are excellent. I'm happy to answer questions after the talk or you can reach me through my website.